Proposta de Projeto: CogniFlow AI

Traduzindo Texto Denso em Aprendizado Visual

1. O Problema

- **Contexto:** Textos longos geram uma sobrecarga cognitiva extrema, afetando especialmente alunos surdos (que possuem um idioma de base visual) e neurodivergentes (TDAH/TEA).
- **Gargalo Atual:** Professores não possuem tempo hábil (em média 3 horas por aula) para adaptar materiais densos manualmente para formatos acessíveis.
- **Por que usar IA Generativa:** Soluções de software com regras fixas não compreendem a semântica do texto. A IA Generativa é estritamente necessária para interpretar conceitos complexos e traduzi-los dinamicamente para um formato visual estruturado (código Mermaid.js), preservando o sentido e as conexões lógicas.

2. Dados e Ingestão

- **Fontes:** PDFs densos, apostilas e transcrições de aulas não-estruturadas.
- **Fluxo de Entrada:** Os documentos passam por um processo de chunking semântico (divididos preferencialmente por cabeçalhos ou tamanho fixo com overlap). Os textos extraídos são convertidos em embeddings vetoriais.
- **Recuperação (RAG):** O contexto relevante é recuperado no banco de dados via busca por similaridade e injetado no prompt do modelo. Isso garante que a geração visual utilize exclusivamente o material didático validado, eliminando dependência de conhecimento externo.

3. Arquitetura Técnica

A estrutura foca em robustez e isolamento de responsabilidades:

- **Backend Orquestrador:** Construído em Python utilizando o framework assíncrono FastAPI, em conjunto com ferramentas de orquestração de IA (como LangChain). A aplicação seguirá padrões rigorosos de Clean Architecture e SOLID.

- **Banco de Dados Vetorial:** PostgreSQL com a extensão `pgvector`, orquestrado via Docker Compose para centralizar dados relacionais e vetoriais.
- **Modelos LLM:** O ecossistema suportará o Gemini (via API) para tirar proveito da grande janela de contexto e baixo custo, com facilidade de alternância para modelos abertos (Llama/Qwen via Ollama) para garantir privacidade total de dados institucionais sensíveis.
- **Frontend:** Interface interativa desenvolvida com Next.js e React Flow. A aplicação utilizará motores de auto-layout (como Dagre) para calcular o espaçamento e a hierarquia dos nós na tela automaticamente, garantindo a renderização visual fluida e acessível do diagrama sem sobrecarregar matematicamente o LLM.

4. Avaliação e Validação

A solução será submetida a um conjunto de testes contendo pelo menos 20 exemplos reais de materiais acadêmicos. A infraestrutura de validação será construída no backend utilizando PyTest e Testcontainers.

- **Métrica Principal (Taxa de Sucesso de Renderização):** Validação estática para garantir que o LLM gerou um JSON estrito perfeitamente validado, sem quebrar a interface.
- **Cobertura (Recall) e Desempenho:** Testes automatizados para cruzar as entidades do texto com os nós do diagrama gerado, além da monitorização do tempo fim-a-fim, visando comprovar a redução da barreira de 3 horas para menos de 2 minutos.

5. Riscos e Cuidados Éticos (LGPD)

- **Alucinação Visual:** O risco do modelo inventar conexões inexistentes no material. Mitigação: Prompts de sistema restritivos ("responda apenas com código baseado no contexto") e validação automatizada que cruza a saída com o texto original do RAG.
- **Vazamento de Dados:** Risco de expor propriedade intelectual das apostilas. Mitigação: Isolamento da base de dados vetorial e capacidade da arquitetura em rodar LLMs de forma local, mantendo conformidade total com a LGPD.
- **Injeção de Prompt:** Manipulação dos inputs do usuário. Mitigação: Camada de sanitização de dados no backend em Python (FastAPI) durante a fase de ingestão do documento, bloqueando instruções anômalas antes de atingirem o motor de inferência.